README - Health Misinformation Modelling (Detection & Propagation) Code Suite

This **README** explains how to run, organise and interpret the supplied health misinformation modelling pipelines. It covers text classification, feature fusion, ELM-informed CNN-LSTM classification, network centrality, ELM-SIRMMM propagation modelling, BERTopic-Virality Prioritisation, and pandemic-related linguistic pattern analysis.
The code is designed for **Google Colab**, but most scripts can also run locally where Python dependencies and the required datasets are available.

1. Important execution rule

The supplied code contains multiple standalone pipelines. Several blocks define their own main() function, constants, helper functions and output folders. Do not paste all blocks into one single Python file and run it as one script unless you deliberately separate them into independent files or notebook sections.

- Use one Colab notebook per pipeline, or save each pipeline as a separate .py file.
- If using one Colab notebook, keep each pipeline in a clearly labelled section and run only the section required.
- If all blocks are merged into one Python file, later definitions may overwrite earlier definitions, and only the final main() block may behave as expected.
- Treat every output folder as pipeline-specific. Do not overwrite outputs from a previous run unless that is intended.

2. Recommended file names

Pipeline	Suggested file	Dataset scope
Text Classification	classification_reproducibility.py	fakeNews.csv, trueNews.csv
Multi-branch fusion model	multi_branch_fusion_monkeypox.py	monkeypox_dataset.csv
ELM-informed CNN-LSTM Classification	elm_cnn_lstm_covid_fnir.py	fakeNews.csv, trueNews.csv
Network centrality	centrality_monant.py	discussion_post.csv
Epidemiological Modelling ELM-SIRMMM Propagation	tri_dataset_elm_sirmmm.py	FibVID, MC-Fake, Monant
BERTopic-Virality Prioritisation	bertopic_vp_tri_dataset.py	COVID--19_FNIR, Constraint, Monkeypox
Pandemic-related Linguistic Pattern Analysis	linguistic_patterns_tri_dataset.py	COVID--19_FNIR, Constraint, Monkeypox

3. Environment setup

The scripts install most packages automatically using pip. In Colab, run the first cell of each pipeline and allow package installation to complete before executing later cells.

- Core scientific stack: pandas, numpy, scikit-learn and scipy.
- Plotting and text analysis: matplotlib, nltk, textblob, textstat and ftfy.
- Deep learning: torch, transformers, accelerate and tensorflow where required.
- Graph analysis: networkx for the centrality pipeline.
- Topic modelling: bertopic, sentence-transformers, umap-learn, hdbscan and gensim where available.

Colab runtime advice: For heavy neural and BERTopic runs, use a GPU runtime where available: Runtime, Change runtime type, Hardware accelerator, GPU.

4. Input datasets

Place input files in the current working directory or in a folder called data/. In Colab, most scripts will prompt for upload if the expected file is not found.

Dataset or input	Used by	Expected file
COVID--19_FNIR fake subset	Text Classification and ELM-informed CNN-LSTM Classification	fakeNews.csv
COVID--19_FNIR true subset	Text Classification and ELM-informed CNN-LSTM Classification	trueNews.csv
Monkeypox labelled dataset	Multi-branch fusion model	monkeypox_dataset.csv
Monant discussion posts	Network centrality	discussion_post.csv
FibVID	Epidemiological Modelling ELM-SIRMMM Propagation	fibvid.csv
MC-Fake	Epidemiological Modelling ELM-SIRMMM Propagation	mc_fake.csv
Monant aggregate or post file	Epidemiological Modelling ELM-SIRMMM Propagation	monant.csv
COVID--19_FNIR harmonised file	BERTopic-Virality Prioritisation and Pandemic-related Linguistic Pattern Analysis	covid19_fnir.csv
Constraint harmonised file	BERTopic-Virality Prioritisation and Pandemic-related Linguistic Pattern Analysis	constraint.csv
Monkeypox harmonised file	BERTopic-Virality Prioritisation and Pandemic-related Linguistic Pattern Analysis	monkeypox.csv

5. Label conventions

The scripts do not all use the same positive label because they were developed for different experimental purposes. Check this before interpreting F1, precision, recall or ROC-AUC.

Pipeline	Label convention
Text Classification	fake = 0, true = 1, primary reporting label is 0.
Multi-branch fusion model	Misinformation class is mapped to 1. The Monkeypox dataset is balanced before splitting.

Pipeline	Label convention
ELM-informed CNN-LSTM Classification	true = 0, fake = 1, positive label is 1.
Network centrality	No binary classifier label is required. The unit is an author node and reply relation.
Epidemiological Modelling ELM-SIRMMM Propagation	Misinformation rows usually match 1, fake, false, misinformation, rumour, unverified or misleading.
BERTopic-Virality Prioritisation	Misinformation labels are harmonised to 1 and factual labels to 0.
Pandemic-related Linguistic Pattern Analysis	Labels are retained where available, but the main comparison is dataset-level linguistic variation.

6. Pipeline 1- Text Classification

Purpose: benchmark conventional machine learning, Word2Vec neural models, frozen DistilBERT models and fine-tuned transformer models for health misinformation text classification.

Inputs: fakeNews.csv and trueNews.csv, both with a Text column.

- Preprocessing variants: baseline unigram TF-IDF, unigram and bigram TF-IDF, fewer stop words, lemmatisation and stemming.
- Conventional classifiers: Naive Bayes, Gradient Boosting, Linear SVM, Decision Tree, Random Forest, Bagging, AdaBoost, SGD logistic loss, Logistic Regression and MLP.
- Neural models: CNN+Word2Vec, LSTM+Word2Vec and CNN-LSTM+Word2Vec.
- Frozen transformer models: CNN+FrozenDistilBERT, LSTM+FrozenDistilBERT and CNN-LSTM+FrozenDistilBERT.
- Fine-tuned transformer models: FineTunedDistilBERT and FineTunedRoBERTa.
- Evaluation metrics: accuracy, primary-class F1, precision, recall, macro F1, weighted F1 and primary-class ROC-AUC where scores are available.

```
python classification_reproducibility.py --run_ml
python classification_reproducibility.py --run_all
python classification_reproducibility.py --run_all --epochs_word 8 --epochs_frozen 3 --epochs_transformers 3
```

- dataset_summary.csv
- split_summary.csv
- conventional_ml_results_raw.csv and conventional_ml_results_percent.csv
- conventional_ml_summary.csv
- word2vec_neural_results_percent.csv
- frozen_distilbert_results_percent.csv
- finetuned_transformer_results_percent.csv
- deep_learning_summary.csv
- hybrid_model_summary.csv
- transformer_model_summary.csv

7. Pipeline 2 - Multi-branch fusion model

Purpose: train a multi-branch model combining transformer text representations with rhetorical, stance, ELM and TPB-inspired psychological features. The pipeline supports ablation experiments by removing selected feature branches.

Input: monkeypox_dataset.csv. The script attempts to detect the text, label and engagement columns automatically. The dataset is randomly undersampled to the minority class before splitting.

- Text branch: DistilBERT sentence representation from distilbert-base-uncased.
- Rhetorical branch: discourse cues, attribution cues, elaboration cues and sensational cues.
- Stance branch: support, denial, query and comment-like signals.
- ELM branch: readability, lexical richness, sentiment, length, punctuation, uppercase and urgency features.
- TPB branch: affective valence, modal verbs, group pronouns, mentions, social comparison, certainty, instructional language and directive hashtags.
- Auxiliary outputs: scaled engagement regression and Cognitive Propagation Score prediction, alongside binary classification.

```
python multi_branch_fusion_monkeypox.py
python multi_branch_fusion_monkeypox.py --skip_ablation
python multi_branch_fusion_monkeypox.py --skip_baselines
```

- dataset_summary.csv
- split_summary.csv
- balance_summary.json and dataset_metadata.json
- baseline_classifier_results.csv
- fusion_model_results.csv
- ablation_results.csv
- propagation_results.csv
- cps_statistics.csv
- test_predictions.csv
- training_history.csv

Interpretation caution: Propagation and CPS outputs are auxiliary prediction targets. They should be interpreted as model-derived prioritisation or ranking signals unless observed diffusion labels are available and explicitly validated.

8. Pipeline 3 - ELM-informed CNN-LSTM Classification

Purpose: compare a base CNN-LSTM classifier against ELM-only, text-plus-ELM and extended feature models using stratified cross-validation.

Inputs: fakeNews.csv and trueNews.csv, both with a Text column.

- Model variants: Base, ELM_Only, Enhanced_Text_ELM and Extended_Text_ELM_Additional.
- ELM central-route features: Flesch-Kincaid Grade, vocabulary richness, sentiment polarity, text length and average words per sentence.
- ELM peripheral-route features: exclamation ratio, question ratio, capitalisation ratio, all-caps count and urgency frequency.

- Additional linguistic features: punctuation ratios, sentence structure ratios, POS distributions, risk terms, attribution and discourse markers.
- Validation design: stratified cross-validation with fold-level outputs, out-of-fold predictions and paired statistical tests.
  python elm_cnn_lstm_covid_fnir.py
- dataset_summary.csv and dataset_summary.json
- runtime_versions.json
- feature_definitions.csv
- fold_level_results.csv
- out_of_fold_predictions.csv
- fold_assignments.csv
- aggregate_metrics_raw.csv and aggregate_metrics_percent.csv
- performance_improvement_vs_base.csv
- statistical_tests.csv
- confusion_matrix_*.csv and confusion_matrix_*.png
- roc_curves_all_variants.png
- mean_performance_by_variant.png

9. Pipeline 4 - Network centrality

Purpose: build a directed reply graph from discussion posts and compute traditional and advanced centrality measures for propagation analysis.

Input: discussion_post.csv. Expected columns include id, author_id, parent_post_id and body or an equivalent text field. Optional fields include published_at, up_votes and down_votes.

- Graph definition: a directed edge is created from the reply author to the parent post author. Edge weight is the number of replies from one author to another.
- Traditional metrics: weighted degree, eigenvector centrality, betweenness centrality and incoming harmonic closeness.
- Advanced metrics: propagation centrality, misinformation vulnerability centrality and dynamic influence centrality.
- Validation proxies: engagement_sum, engagement_mean, emotion_proportion and emotion_count are used for rank alignment checks.
- Intervention logic: node removal summaries estimate the proportional reduction in total edge weight after removing top-ranked nodes.
  python centrality_monant.py --discussion discussion_post.csv --top_k 10
- dataset_summary.csv
- graph_summary.csv
- author_proxy_features.csv
- all_centrality_scores.csv
- top_nodes_by_metric.csv
- traditional_centrality_top_nodes.csv
- advanced_centrality_top_nodes.csv

- metric_overlap_summary.csv
- traditional_advanced_union_summary.csv
- proxy_alignment_summary.csv
- node_removal_intervention_summary.csv

10. Pipeline 5 - Epidemiological Modelling ELM-SIRMMM Propagation

Purpose: fit fixed-rate SIRMMM and ELM-conditioned SIRMMM models across FibVID, MC-Fake and Monant. The experiment tests whether sentiment, engagement and cognition provide enough temporal signal to modulate the misinformation transmission rate.

Inputs: fibvid.csv, mc_fake.csv and monant.csv.

- Model compartments: MS means misinformation susceptible, MI means active misinformation sharers and MR means misinformation recovered or disengaged.
- Daily signals: misinformation incidence, sentiment, engagement and cognition are aggregated by date.
- Plain SIRMMM: estimates a fixed misinformation transmission rate and recovery or disengagement rate.
- ELM-SIRMMM: estimates beta_m(t) as a time-varying function of sentiment, engagement and cognition.
- Robustness checks: log1p engagement, min-max scaling, raw engagement, binary activity and alternative feature scaling variants.

  beta_m(t) = beta0 + beta_sentiment * sentiment(t) + beta_engagement * engagement(t) + beta_cognition * cognition(t)

  python tri_dataset_elm_sirmmm.py
- Each dataset folder: column_mapping.json, dataset_summary.csv, prepared_rows.csv and daily_aggregated.csv.
- Model outputs: performance_summary.csv, trajectory_summary.csv, elm_coefficients.csv, daily_model_outputs.csv and robustness_checks.csv.
- Combined outputs: all_performance_summary.csv, all_trajectory_summary.csv, all_elm_coefficients.csv, all_robustness_checks.csv, report_fit_table.csv, report_elm_trajectory_table.csv and report_elm_coefficients_wide.csv.
- Plots: observed versus fitted trajectories, compartment trajectories, beta_m(t) and normalised behavioural inputs.

Interpretation caution: Fitted ELM coefficients are conditional associations inside the model. They should not be interpreted as causal effects unless supported by additional experimental or quasi-experimental evidence.

11. Pipeline 6- BERTopic-Virality Prioritisation

Purpose: perform topic modelling and virality prioritisation across COVID--19_FNIR, Constraint and Monkeypox.

Correct dataset scope: this pipeline uses COVID--19_FNIR, Constraint and Monkeypox only. It does not use MC-Fake, FibVID or Monant.

- Monkeypox uses observed engagement, defined as log(1 + available likes + retweets + replies + quotes).

- COVID--19_FNIR and Constraint use a transferred proxy signal from a logistic propensity model trained on Monkeypox.
- Proxy VP should be interpreted as a within-dataset ranking signal, not as observed engagement magnitude.
- Topic modelling uses SentenceTransformer embeddings, UMAP dimensionality reduction, HDBSCAN clustering and BERTopic topic representation.
- Evaluation includes clustering metrics, topic coherence where available, VP flagged cluster counts and representative high-priority documents.
  E_i = log(1 + likes + retweets + replies + quotes)
  python bertopic_vp_tri_dataset.py
- Dataset outputs: *_prepared_scored.csv, *_documents_with_topics_and_vp.csv, *_cluster_vp_summary.csv, *_vp_flagged_cluster_counts.csv, *_clustering_metrics.csv, *_coherence.csv and *_representative_vp_topics.csv.
- Combined outputs: all_cluster_vp_summaries.csv, all_vp_flagged_cluster_counts.csv, all_clustering_metrics.csv, all_coherence_results.csv, all_representative_vp_topics.csv, propensity_model_training_note.json and column_mappings.json.
- Archive output: bertopic_vp_covid_constraint_monkeypox_results.zip.

12. Pipeline 7 - Pandemic-related Linguistic Pattern Analysis

Purpose: compare linguistic and rhetorical patterns across COVID--19_FNIR, Constraint and Monkeypox. It extracts readability, punctuation, persuasive or fear-related lexicon, sentiment, lexical diversity and priority-ranking signals.

Correct dataset scope: this pipeline uses COVID--19_FNIR, Constraint and Monkeypox only. It does not use MC-Fake, FibVID or Monant.

Inputs: covid19_fnir.csv, constraint.csv and monkeypox.csv.

- Readability: Flesch Reading Ease and Flesch-Kincaid Grade Level.
- Rhetorical punctuation: exclamation count and rate, question count and rate, and exclamation-to-question ratio.
- Persuasive or fear-related lexicon: urgent, emergency, fear, panic, alarming, crisis, warning and disaster.
- Other linguistic features: sentiment polarity, type-token ratio, uppercase ratio, token count and character count.
- Priority signal: observed engagement where native fields exist, otherwise a Monkeypox-trained engagement-propensity logit.
- Inferential analysis: normality checks, Kruskal-Wallis tests, Dunn post hoc tests and readability consistency via Spearman correlation.
- Robustness checks: length-restricted analysis and minimal-cleaning punctuation summaries.
  python linguistic_patterns_tri_dataset.py
- posts_with_linguistic_features.csv
- dataset_summary.csv
- feature_summary_by_dataset.csv
- normality_checks.csv
- kruskal_wallis_results.csv

- dunn_posthoc_bonferroni_results.csv
- spearman_readability_consistency.csv
- engagement_availability_report.csv
- engagement_propensity_model_report.csv
- high_priority_qualitative_examples.csv
- robustness_length_restriction_info.csv
- robustness_length_restricted_summary.csv
- robustness_length_restricted_kruskal.csv
- robustness_length_restricted_dunn.csv
- robustness_minimal_cleaning_rhetorical_summary.csv
- pipeline_config.json
- analysis_summary.txt
- linguistic_patterns_results.zip

13. Reproducibility checklist

- The correct dataset files were used and the expected text, label, date and engagement columns were detected.
- Label mappings match the intended positive class for the relevant pipeline.
- The random seed is fixed at 42 unless deliberately changed.
- The train-test split, validation split or cross-validation design is recorded.
- Any balancing or undersampling step is reported.
- Engagement proxies are clearly distinguished from observed engagement.
- Output files are saved, named consistently and backed up.
- The runtime environment is recorded where available.
- Preprocessing, feature scaling and vectorisation are fitted only on training data where applicable.

14. Common issues and fixes

Issue	Likely cause	Fix
File not found	Expected files are not in the working directory or data/ folder.	Place files in the current working directory or data/. In Colab, rerun the upload cell and confirm the filename.
Wrong text column detected	The dataset uses a column name outside the candidate list.	Edit the relevant text_candidates list or manually rename the column in the CSV.
Wrong label mapping	The label values differ from those expected by the script.	Inspect unique values in the label column and update the mapping function or label-value sets.
CUDA out of memory	Batch size, sequence length or model size is too large.	Reduce BATCH_SIZE, reduce MAX_LEN, run fewer epochs or use a smaller model.

Issue	Likely cause	Fix
BERTopic takes too long	The dataset is large or the runtime has limited memory.	Start with a sample, verify column detection and VP scoring, then run the full dataset.
Very high or unstable performance	Possible duplicates, leakage, near-duplicates or inconsistent labels.	Check duplicates, split leakage, label consistency and whether preprocessing was fitted only on training folds.
Engagement proxy confusion	Observed engagement and proxy propensity are being interpreted as equivalent.	Report observed engagement separately from model-derived propensity scores.

15. Recommended output archive structure

```
project_outputs/
  text_classification/
  fusion_monkeypox/
  elm_cnn_lstm_classification/
  centrality_monant/
  elm_sirmmm_tri_dataset/
  bertopic_virality_prioritisation/
  pandemic_related_linguistic_pattern_analysis/
  README_health_misinformation_code_suite_regenerated.docx
  README_health_misinformation_code_suite_regenerated.pdf
```

16. Minimal run order

1. Run Pipeline 1, Text Classification, to establish baseline detection performance.
2. Run Pipeline 3, ELM-informed CNN-LSTM Classification, to test theory-informed feature enhancement.
3. Run Pipeline 2, Multi-branch fusion model, to test branch-level contribution and ablation.
4. Run Pipeline 4, Network centrality, to analyse network influence and vulnerability.
5. Run Pipeline 5, Epidemiological Modelling ELM-SIRMMM Propagation, to examine temporal propagation dynamics.
6. Run Pipeline 6, BERTopic-Virality Prioritisation, to identify prioritised topic clusters.
7. Run Pipeline 7, Pandemic-related Linguistic Pattern Analysis, to explain cross-dataset rhetorical and readability differences.

17. Final interpretation guide

Use text classification outputs to answer whether models can classify misinformation reliably. Use ablation outputs to assess which feature branches contribute most to classification and auxiliary prediction. Use centrality outputs to identify influential or vulnerable actors in networked discussion. Use ELM-SIRMMM outputs to assess whether behavioural signals improve temporal fit over a fixed-rate baseline. Use BERTopic-Virality Prioritisation outputs to identify topic clusters with higher observed or predicted prioritisation. Use pandemic-related linguistic pattern

outputs to explain how readability, rhetorical style and affective lexicon differ across COVID--19_FNIR, Constraint and Monkeypox.

The strongest reporting position is to separate observed findings from proxy-based or model-derived findings. This keeps the analysis reproducible, interpretable and methodologically defensible.